

Week 3 Interview Prep Guide

Structure:

1. **Round 1: Fundamentals (Beginner → Intermediate)**
 2. **Round 2: Hands-On / Scenario-Based**
 3. **Round 3: Senior-Level / Architecture & Debugging**
 4. **Bonus: 1 Page Cheat Sheet for students**
 5. **Expected answers for every question**
-

ROUND 1: FUNDAMENTALS (WHAT THEY SHOULD KNOW AFTER COREOPS)

1. What is Pod Security Admission (PSA)? Why was it introduced?

Expected answer:

- PSA is the built-in Kubernetes replacement for PodSecurityPolicy (PSP).
- It defines 3 levels: **Privileged, Baseline, Restricted**.
- It blocks workloads with risky securityContext fields.
- Enforced at **namespace level**.

Follow-up: “How did we test PSA in the lab?”

→ We applied Restricted to a namespace and deployed a privileged pod → got blocked.

2. What are seccomp profiles? Why do they matter?

Expected answer:

- Seccomp filters syscalls the container is allowed to use.
- Default profile: runtime/default.
- It reduces kernel attack surface.
- Applied via pod annotations or securityContext.

Follow-up: “How did we verify seccomp is applied?”

→ We used `kubectl get pod -o yaml | grep seccomp`.

3. AppArmor vs Seccomp; what's the difference?

Expected answer:

- **Seccomp** filters syscalls.
 - **AppArmor** applies file system & process behavior restrictions (profiles).
 - Both used together for defense-in-depth.
-

4. What is RBAC and why is it critical?

Expected answer:

- RBAC controls **who can do what** in the cluster.
- Uses: Role, ClusterRole, RoleBinding, ClusterRoleBinding.
- It enforces least privilege access.

Follow-up: “How did we test RBAC permissions?”

→ `kubectl auth can-i list pods --as=username`.

5. Difference between Role vs ClusterRole?

Expected answer:

- **Role** → namespace scoped
 - **ClusterRole** → cluster-scoped or reusable across namespaces
-

6. How do service accounts authenticate to the API server?

Expected answer:

- ServiceAccounts get a **projected token**, mounted at
`/var/run/secrets/kubernetes.io/serviceaccount/token`
- Pod uses that JWT for API requests.

Follow-up: "What lab proved this?"

→ We exec'd into a pod and listed SA tokens.

7. What is a SealedSecret?

Expected answer:

- A CRD that allows storing **encrypted secrets in Git**.
- Even cluster admins cannot read sealed secrets.
- Controller decrypts them using a private key inside the cluster.

Follow-up: "What real-world problem does it solve?"

→ Secret sprawl in GitOps workflows.

8. Why are plain Kubernetes Secrets insecure?

Expected answer:

- They are only base64 encoded.
 - Stored unencrypted unless KMS enabled.
 - Anyone with etcd access can read them.
-

9. What is Cosign and why is it important?

Expected answer:

- Tool from Sigstore to **sign & verify container images**.
- Ensures provenance: you know who built the image.
- Prevents supply chain attacks (tampered images).

Follow-up: "What did we do in lab?"

→ Signed nginx and verified it.

THE DEVOPS WAR ROOM

ROUND 2: HANDS-ON / PRACTICAL SCENARIOS

10. A developer says, "My pod was rejected. It works locally." What's your first move?

Expected answer:

1. Check namespace labels: `kubectl get ns --show-labels`
2. Describe pod: `kubectl describe pod`
3. Look for the **PSA violation** message.

Follow-up: "What fields usually violate Restricted PSP?"

→ privileged, hostPID, hostNetwork, capabilities, hostPath...

11. You created an RBAC RoleBinding, but the user still cannot list pods. Why?

Possible reasons:

- Wrong namespace
- Wrong service account
- Using ClusterRoleBinding by mistake
- You must switch to that user:

```
kubectl auth can-i list pods --as=system:serviceaccount:ns:sa
```

12. How can you test what a ServiceAccount can do without deploying a pod?

```
kubectl auth can-i get pods --as=system:serviceaccount:ns:sa
```

13. Your cluster has many secrets. How to audit who can read them?

```
kubectl auth can-i get secret --as=username -A
```

And: `kubectl get clusterrole -o yaml | grep -i secrets -n`

14. Your application uses a token that expired (1-hour lifetime). How do you debug?

Expected:

1. Exec into pod
 2. Check projected service account token
 3. Validate JWT expiry (exp)
 4. Check API server projected token settings
-

15. A GitOps pipeline applies SealedSecret but it doesn't decrypt. How do you debug?

Checklist:

- Is sealed-secrets-controller running?
 - Check logs: `kubectl logs -n kube-system deploy/sealed-secrets-controller`
 - Was the SealedSecret encrypted using the correct *public key*?
 - Was the namespace correct?
-

ROUND 3: ADVANCED / ARCHITECTURAL (SENIOR LEVEL)

16. Why is PSA considered "shift-left security"?

Expected:

- It blocks insecure workloads at admission time.
 - Prevents security drift.
 - Lightweight compared to heavy PSP.
-

17. How do you secure etcd in a Kubernetes cluster?

Expected:

- Enable TLS
 - Enable encryption at rest
 - Restrict access (etcd lives only on control-plane nodes)
 - Use dedicated disks and automatic snapshots
 - No external access except API server
-

18. Why is workload identity replacing static service account tokens?

Expected:

- SA tokens are long-lived → security risk
 - Workload Identity integrates with cloud IAM
 - Temporary credentials
 - Auto-rotated
 - Least privilege per workload
-

19. Describe a real-world supply chain attack on container registries.

Expected:

- Attackers push malicious image with same tag
- Developers pull updated latest unknowingly
- Signing (Cosign) + policy prevents execution of unsigned images

THE DEVOPS WAR ROOM

Follow-up: “How do you enforce only signed images run?”

→ Gatekeeper / Kyverno policies.

20. Explain the difference between image signing and attestation.

Expected:

- **Signing** → verifies authenticity of image
 - **Attestation** → metadata: build time, SLSA level, CI pipeline info
-

BONUS: 1-PAGE CHEAT SHEET

Pod Security

Restricted mode blocks:

- privileged
 - hostNetwork, hostPID, hostIPC
 - additional capabilities
 - hostPath volumes
 - readOnlyRootFilesystem=false
-

RBAC

- Role → namespace
 - ClusterRole → cluster
 - RoleBinding → binds Role to SA
 - Debug with: `kubectl auth can-i`
-

Service Accounts

- Mounted at `/var/run/secrets/...`
 - Token is a JWT
 - Used for in-cluster authentication
-

Secrets

- Base64 only
 - Secure with:
 - KMS encryption
 - SealedSecrets
 - External Secrets Operator
-

Supply Chain

- Cosign sign
- Cosign verify
- Produce SBOM
- Enforce with Kyverno/Gatekeeper

